Esercitazione S2

Luca Calzia -16/10/2026

1. Analisi ed Obiettivo

Dato uno script python, effettuare il debugging senza runnare il codice.
Individuare eventuali casi particolari o input imprevisti non gestiti e identificare possibili errori di sintassi e di logica, proponendo per ciascun problema una soluzione motivata.

```python
import datetime

while True
    comando_utente = input("Cosa vuoi sapere? ")
    if comando_utente == "esci":
        print("Arrivederci!")
    break
    else:
        print(assistente_virtuale(comando_utente))

def assistente_virtuale(comando):
    if comando == "Qual è la data di oggi?":
        oggi = datetime.datetoday()
        risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
    elif comando == "Che ore sono?":
        ora_attuale = datetime.datetime.now().time()
        risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
    elif comando == "Come ti chiami?":
        risposta = "Mi chiamo Assistente Virtuale"
    else:
        risposta = "Non ho capito la tua domanda."
```

2. Svolgimento

In seguito elencherò gli errori del codice, la riga su cui si trovano e la soluzione proposta:

Riga 3

```
3   while True
```

Mancano i due punti da prassi per il ciclo while.
Codice corretto:

```
3   while True:
```

Riga 7

```
5       if comando_utente == "esci":
6           print("Arrivederci!")
7       break
8       else:
```

E' sbagliata l'indentazione del *break*, deve stare dentro le istruzioni dell'if.
Codice corretto:

```
5       if comando_utente == "esci":
6           print("Arrivederci!")
7           break
8       else:
```

Riga 9

```
9           print(assistente_virtuale(comando_utente))
```

La funzione assistente_virtuale(comando) viene definita dopo aver chiamato la funzione quindi risulta come non definita. Deve essere inserita prima.
Codice corretto:

```
import datetime

def assistente_virtuale(comando):
    # istruzioni
    return

while True:
    comando_utente = input("Cosa vuoi sapere? ")
    if comando_utente == "esci":
        print("Arrivederci!")
        break
    else:
        print(assistente_virtuale(comando_utente))
```

Riga 13

```
12      if comando == "Qual è la data di oggi?":
13          oggi = datetime.datetoday()
14          risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
```

La funzione *datetime.datetoday()* non esiste nella libreria datetime, la funzione corretta è *datetime.date.today()*.
Codice corretto:

```
12      if comando == "Qual è la data di oggi?":
13          oggi = datetime.date.today()
14          risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
```

3. Ottimizzazione del codice proposta

```
import datetime

def assistente_virtuale():
    lista_stringa_utente = input("Cosa vuoi sapere? ").lower().strip("?!.,").split()
    match lista_stringa_utente:
        case _ if ( "data" in lista_stringa_utente or "giorno" in lista_stringa_utente) and any(p in lista_stringa_utente for p in ["ora", "ore", "orario"]):
            print(f"Il giorno e l'ora di adesso sono: {datetime.date.today().strftime("%d/%m/%Y")} {datetime.datetime.now().time().strftime("%H:%M")}")
        case _ if "data" in lista_stringa_utente or "giorno" in lista_stringa_utente:
            print(f'La data di oggi è {datetime.date.today().strftime("%d/%m/%Y")}')

        case _ if any(p in lista_stringa_utente for p in ["ora", "ore", "orario"]):
            print(f"L'ora attuale è {datetime.datetime.now().time().strftime("%H:%M")}")

        case _ if "nome" in lista_stringa_utente or "chiami" in lista_stringa_utente or "chi sei" in " ".join(lista_stringa_utente):
            print("Mi chiamo Assistente Virtuale")

        case _ if any(p in lista_stringa_utente for p in ["esci", "termina", "chiudi"]):
            return

        case _:
            print("Non ho capito la domanda.")
            assistente_virtuale()
    return
```

Il programma è strutturato all'interno della funzione *assistente_virtuale()*, che acquisisce la domanda dell'utente tramite *input()* e la prepara per il riconoscimento delle parole chiave: *lower()* converte il testo in minuscolo, *strip()* rimuove la punteggiatura indicata dall'inizio e dalla fine della stringa e *split()* divide la frase in una lista di parole. Successivamente, una struttura *match-case* verifica le condizioni nell'ordine ed esegue il primo caso corrispondente. Il primo caso riconosce una richiesta contenente sia parole relative alla data sia parole relative all'orario e stampa entrambe le informazioni; i casi successivi gestiscono separatamente la data, l'ora, il nome dell'assistente e i comandi di uscita. La funzione *any()* permette di verificare la presenza di almeno una parola tra quelle previste, mentre *join()* ricompone la frase per riconoscere l'espressione "chi sei". Il modulo *datetime* fornisce la data e l'ora correnti, che vengono formattate attraverso *strftime()* e stampate. Se la domanda non viene riconosciuta, il programma mostra un messaggio e richiama la funzione stessa per consentire un nuovo tentativo, utilizzando la ricorsione. Dopo una risposta valida o un comando di uscita, la funzione termina.